9. El plazo de entrega se divide en dos momentos:
a. El día 09 de octubre para adjuntar la evidencia del proceso de análisis y diseño de la solución (Diagrama de clase incluido).

Contextualización: Para esta actividad se pide simular una versión inicial de una plataforma social con funcionalidades para agregar amigos, enviar solicitudes de amistad, reaccionar a fotos de otras personas, etc… Para ello se pide utilizar lenguaje C++, punteros, memoria dinámica, y programación orientada a objetos; procurando siempre escoger las variables y el diseño adecuado para cumplir con todas las funcionalidades que esta versión de la plataforma tendrá y sentar algunas bases para la próxima.

Análisis: Para este desafío deberemos leer y escribir en archivos, sin embargo leerlos completos puede resultar muy costoso y a veces innecesario. Se implementará una estrategia donde se separe la información que requiere cada sección del programa en archivos diferentes. Ya que en ocasiones no toda la información se necesitará que se cargue para la acción específica que requiere el usuario, pero, en caso de haberse cargado ya, se quedará así. Solo se liberará la memoría cuando hayan cambios en los datos (teniendo ahora un arreglo que contenga toda nueva información) que requieran ser guardados en ese instante. Un ejemplo es, el usuario inicia sesión, quiere aceptar o enviar una solicitud de amistad: en este punto, es innecesario para el programa cargar los archivos de mensajes, ya que esta información no es requerida por el momento para la acción que el usuario quiere realizar.

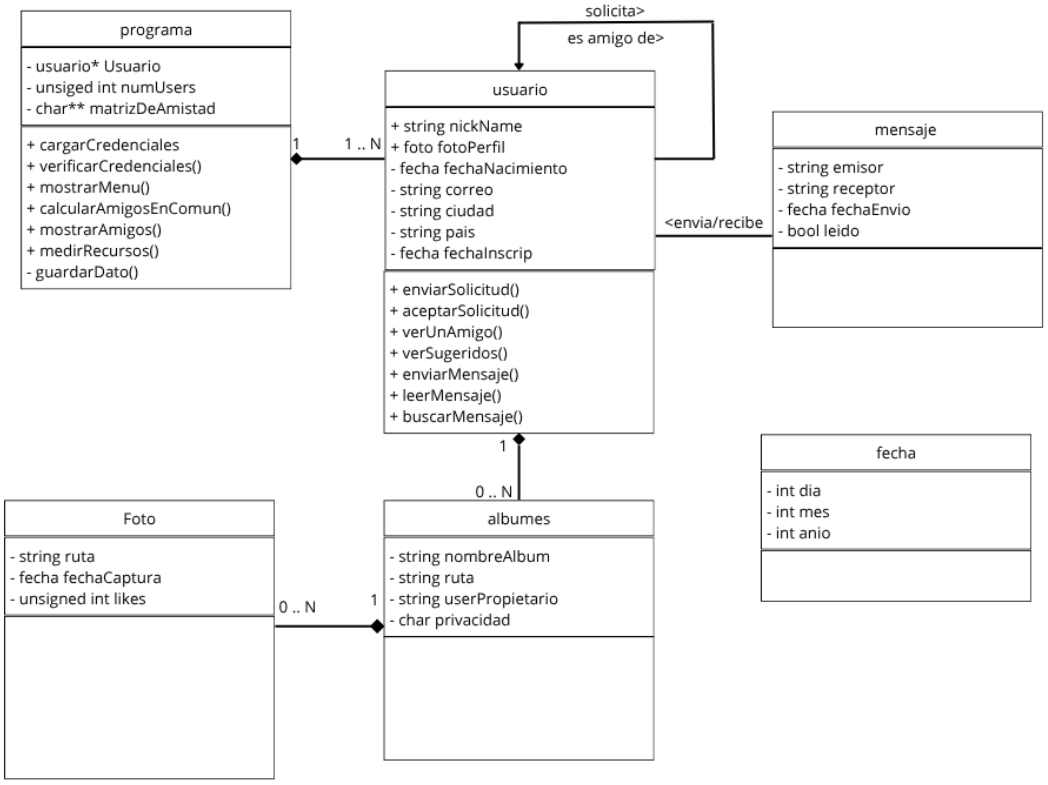
Los datos que cada usuario tendrá son: nombre de usuario, fecha de nacimiento, correo, ruta de su foto de perfil, ciudad, país, hora de inscripción a la plataforma, una lista de 0 a n amigos, una lista de mensajes que ha enviado, una lista de mensajes que ha recibido y una lista de solicitudes de amistad.

Las sugerencias de amistad se pueden calcular durante la ejecución para ese usuario específico en base a los amigos en común (uso de la matriz de adyacencia). Los álbumes y las fotos son clases aparte.

Consideraciones: Para la matriz de adyacencia que indica quien es amigo de quien, se entiende que es una matriz simétrica con ceros en su diagonal (estos ceros no aportan información útil) y unos en donde haya una relación de amigos (esto se sabrá al leer la lista de amigos de cada usuario), por lo cual se puede pensar en solo tener en cuenta la sección triangular superior de esta matriz, lo cual nos ahorra procesos innecesarios para generar y leer todos los datos de esta matriz, además de consumir un 50% menos de memoria de lo que consumiría la matriz original.

En el desarrollo del proyecto se intentará minimizar al máximo la cantidad de memoria usada. Por ejemplo, representando la matriz anterior no como una matriz de chars, si no usando operaciones a nivel de bits, así como en el desafío 1, donde cada 3 bits representaba un estado.

Propuesta:


programa
- usuario* Usuario
- unsiged int numUsers
- char** matrizDeAmistad
+ cargarCredenciales
+ verificarCredenciales()
+ mostrarMenu()
+ calcularAmigosEnComun()
+ mostrarAmigos()
+ medirRecursos()
- guardarDato()
1
1 .. N
solicita>
es amigo de>
usuario
+ string nickName
+ foto fotoPerfil
- fecha fechaNacimiento
- string correo
- string ciudad
- string pais
- fecha fechaInscrip
+ enviarSolicitud()
+ aceptarSolicitud()
+ verUnAmigo()
+ verSugeridos()
+ enviarMensaje()
+ leerMensaje()
+ buscarMensaje()
<envia/recibe
mensaje
- string emisor
- string receptor
- fecha fechaEnvio
- bool leido
1
0 .. N
fecha
- int dia
- int mes
- int anio
Foto
- string ruta
- fecha fechaCaptura
- unsigned int likes
0 .. N
1
albumes
- string nombreAlbum
- string ruta
- string userPropietario
- char privacidad